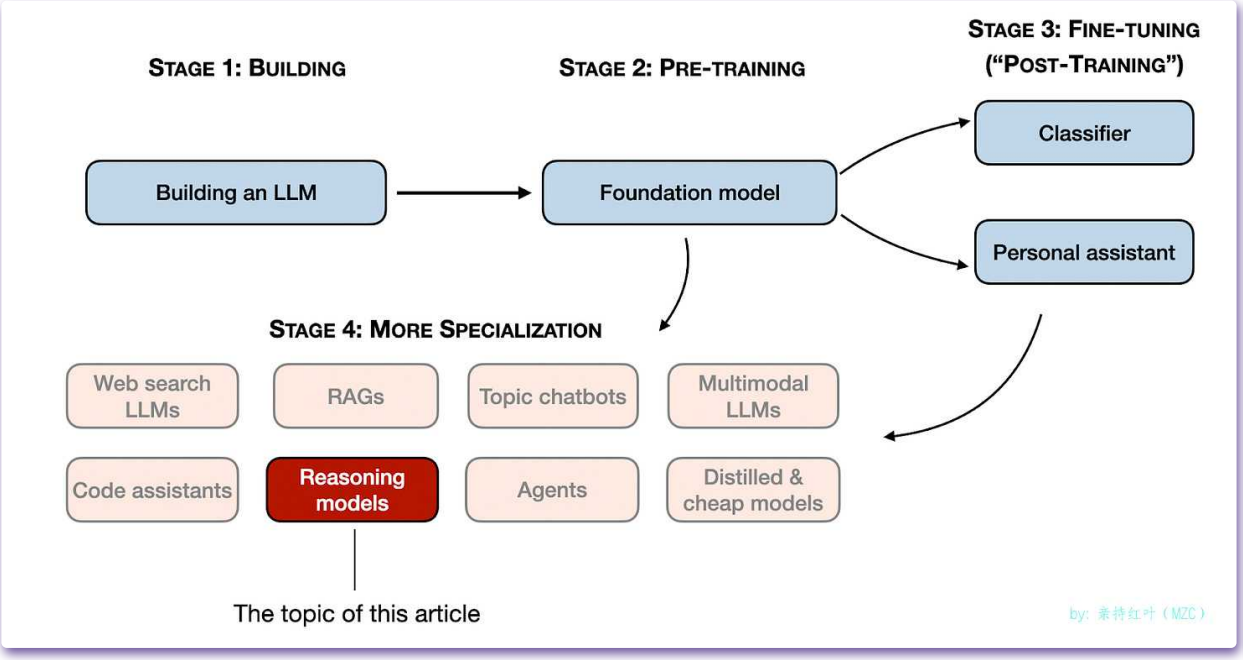
1 Understanding Reasoning LLms

- **原文**

> https://magazine.sebastianraschka.com/p/understanding-reasoning-llms

- 构建和完善推理模型的方法和策略
- 本文介绍了构建推理模型的四种主要方法，或者说如何增强LLM的推理能力。希望本文能提供宝贵的见解，并帮助您应对围绕该主题快速发展的文献和炒作。

  2024 年，LLM领域的专业化程度不断提升。除了预训练和微调之外，我们还见证了从 RAG 到代码助手等专业化应用的兴起。我预计这一趋势将在 2025 年加速，并更加重视针对特定领域和应用的优化（即“专业化”）



(第 1-3 阶段是开发 LLM 的常见步骤。第 4 阶段针对特定用例开发 LLM。)

推理模型的开发是这些专业化方向之一。这意味着我们会对LLM进行改进，使其能够胜任那些最好通过中间步骤解决的复杂任务，例如谜题、高等数学和编程挑战。然而，这种专业化并不能取代其他 LLM 应用。因为将 LLM 转化为推理模型也会带来一些缺点，我将在稍后讨论。

为了让您简要了解下文所涵盖的内容，在本文中，我将：

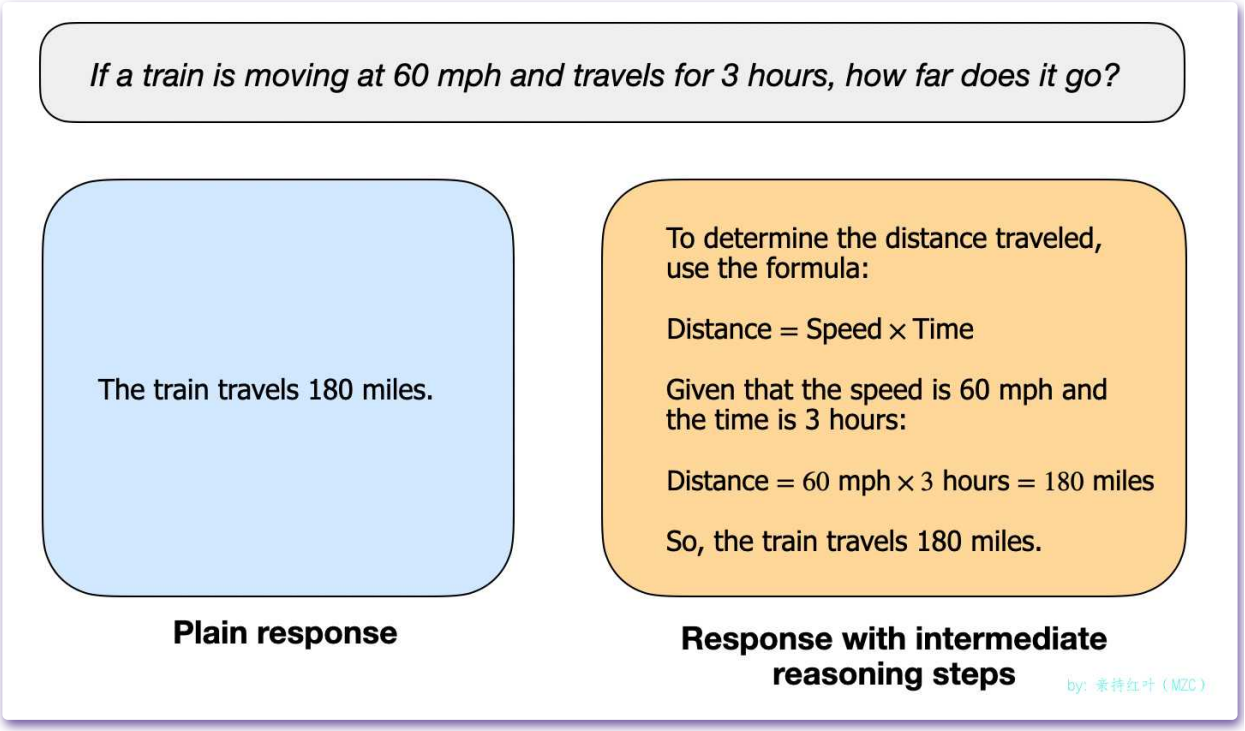
1. 解释“推理模型”的含义
2. 讨论推理模型的优缺点
3. 概述 DeepSeek R1 背后的方法
4. 描述构建和改进推理模型的四种主要方法
5. 分享对 DeepSeek V3 和 R1 发布后 LLM 前景的看法
6. 提供在预算紧张的情况下开发推理模型的技巧

希望你在人工智能今年持续迅猛发展的背景下，发现这篇文章对你有所帮助！

1.1 How do we define "Reasoning Model"

- 如果你从事人工智能（或一般机器学习）领域的工作，你可能对那些模糊且备受争议的定义并不陌生。“推理模型”这个术语也不例外。最终，有人会在一篇论文中正式定义它，但在下一篇论文中又会重新定义它，如此反复。

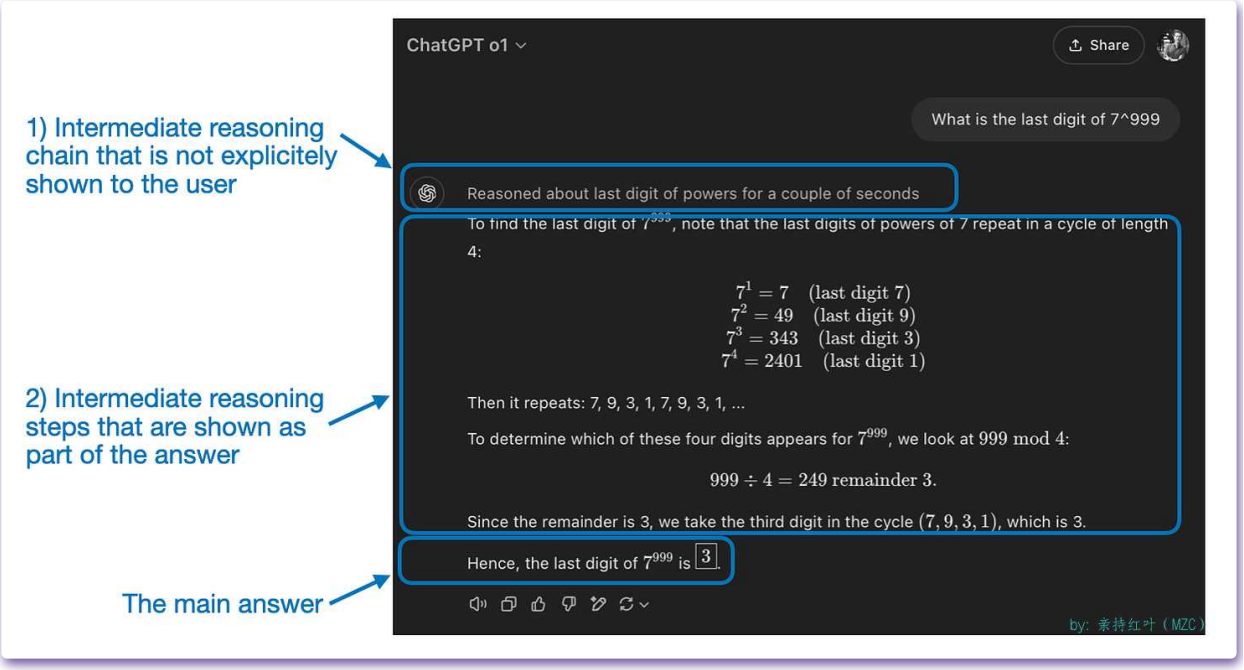
  在本文中，我将“推理”定义为回答那些需要复杂、多步骤生成且包含中间步骤的问题的过程。例如，像“法国首都是哪里？”这样的事实性问答不需要推理。相比之下，像“如果一列火车以 60 英里/小时的速度行驶 3 小时，它能行驶多远？”这样的问题则需要一些简单的推理。例如，它需要识别距离、速度和时间之间的关系，才能得出答案。



普通的LLM可能只提供简短的答案（如左图所示），而推理模型通常包含揭示部分思维过程的中间步骤。（请注意，许多并非专门为推理任务开发的LLM也可以在其答案中提供中间推理步骤。）

大多数现代LLM 都具备基本的推理能力，可以回答诸如“如果一列火车以 60 英里/小时的速度行驶 3 个小时，它能行驶多远？”之类的问题。因此，今天，当我们提到推理模型时，我们通常指的是那些擅长更复杂的推理任务的LLM，例如解决谜题、谜语和数学证明。
此外，如今大多数被贴上推理模型标签的 LLM 都包含一个 “thought” 或 “thinking” 的过程作为其反应的一部分。LLM是否以及如何真正“思考”则另当别论。

推理模型中的中间步骤可以以两种方式出现。首先，它们可能明确包含在响应中，如上图所示。其次，一些推理 LLM（例如 OpenAI 的 o1）会运行多次迭代，其中包含不向用户显示的中间步骤。



“推理”用于两个不同的层面：1）处理输入并通过多个中间步骤生成；2）提供某种推理作为对用户的响应的一部分。

1.2 When should we use reasoning models?

- 现在我们已经定义了推理模型，我们可以进入更有趣的部分：如何构建和改进用于推理任务的 LLM。然而，在深入探讨技术细节之前，重要的是要考虑何时真正需要推理模型。我们什么时候需要推理模型？推理模型被设计用于解决复杂任务，如解决谜题、高级数学问题和具有挑战性的编程任务。然而，对于更简单的任务，如总结、翻译或基于知识的问题回答，它们并非必要。事实上，在所有情况下使用推理模型可能效率低下且成本高昂。例如，推理模型通常使用成本更高、更冗长，有时由于 “过度思考” 而更容易出错。这里也适用一个简单的规则：为任务使用正确的工具 (或 LLM 类型)。
- 下图总结了推理模型的关键优势和局限性。

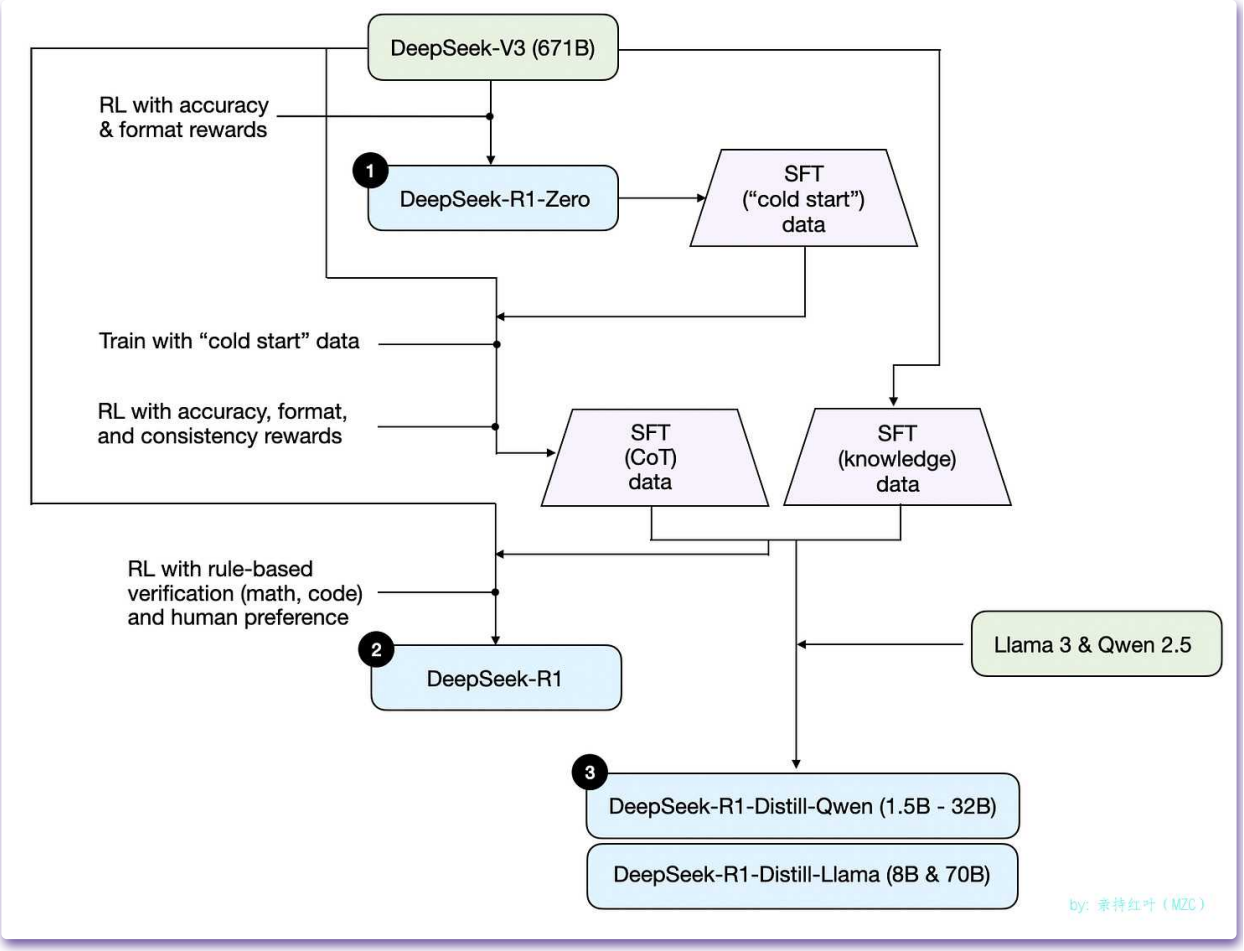
Good at	Bad at
+ Deductive or inductive reasoning (e.g., riddles, math proofs)	– Fast and cheap responses (more inference time)
+ Chain-of-thought reasoning (breaking down multi-step problems)	– Knowledge-based tasks (hallucination)
+ Complex decision-making tasks	– Simple tasks (“overthinking”)
+ Better generalization to novel problems	

by: 亲持红叶 (MZC)

1.3 A brief look at the DeepSeek training pipeline

- 在下一节讨论构建和改进推理模型的四种主要方法之前，我想简要概述一下 DeepSeek R1 流程，具体内容请参阅 **DeepSeek R1 技术报告** 。该报告既是一个有趣的案例研究，也是开发推理 LLM 的蓝图。

  请注意，DeepSeek 并没有发布单一的 R1 推理模型，而是推出了三种不同的变体：DeepSeek-R1-Zero、DeepSeek-R1 和 DeepSeek-R1-Distill。



  DeepSeek R1 技术报告中讨论了 DeepSeek 的三种不同推理模型的开发过程。

  接下来，我们先简单回顾一下上图所示的流程。下一节将详细介绍构建和改进推理模型的四种主要方法。

  1. **DeepSeek-R1-Zero：** 该模型基于 2024 年 12 月发布的 671B 预训练 DeepSeek-V3 基础模型。研究团队使用强化学习 (RL) 和两种奖励类型对其进行训练。这种方法被称为“冷启动”训练，因为它不包含监督微调 (SFT) 步骤，而这通常是基于人工反馈的强化学习 (RLHF) 的一部分。
  2. **DeepSeek-R1：** 这是 DeepSeek 的旗舰推理模型，基于 DeepSeek-R1-Zero 构建。团队通过增加 SFT 阶段和进一步的强化学习训练对其进行了进一步完善，从而在“冷启动”的 R1-Zero 模型上实现了改进。
  3. **DeepSeek-R1-Distill：** 利用前面步骤生成的 SFT 数据，DeepSeek 团队对 Qwen 和 Llama 模型进行了微调，以增强其推理能力。虽然此过程并非传统意义上的“蒸馏”，但它涉及使用较大的 DeepSeek-R1 671B 模型的输出来训练较小的模型（Llama 8B 和 70B，以及 Qwen 1.5B-30B）。

1.4 The 4 main ways to build and improve reasoning models

- 在本节中，我将概述当前用于增强 LLM 推理能力和构建专门推理模型（如 DeepSeek-R1、OpenAI 的 o1 和 o3 等）的关键技术。

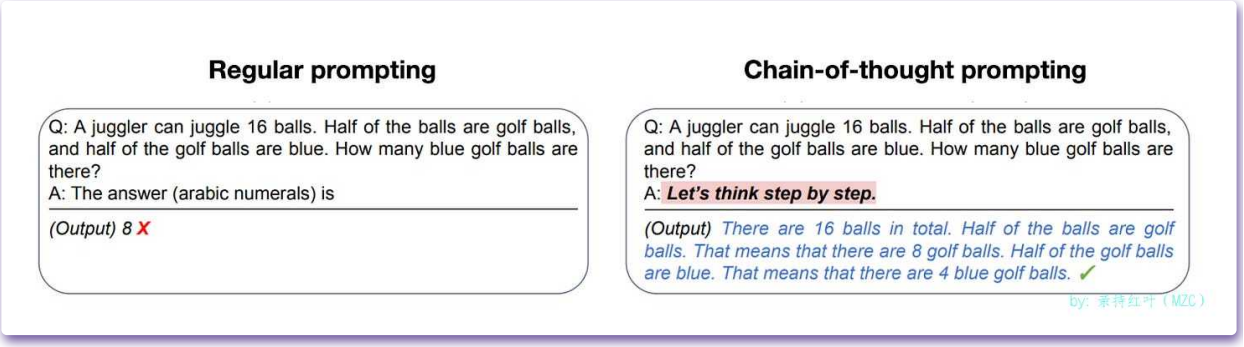
  注：o1 和 o3 的具体工作原理在 OpenAI 之外仍属未知。不过，据传它们将结合推理和训练技术。

1.4.1 Inference-time-scaling

- 提升LLM 推理能力（或任何一般能力）的一种方法是推理时间扩展。该术语可能有多种含义，但在本文中，它指的是在推理过程中增加计算资源以提高输出质量。

  粗略地类比一下，当人类有更多时间思考复杂问题时，往往会做出更好的回答。同样，我们可以运用一些技巧，鼓励LLM在生成答案时更多地“思考”。（LLM 是否真的会“思考”则是另一个话题。）

  推理时间扩展的一个直接方法是巧妙的提示设计。一个经典的例子是 *思路链 (CoT) 提示* ，在输入提示中包含诸如“逐步思考”之类的短语。这鼓励模型生成中间推理步骤，而不是直接跳到最终答案，这通常（但并非总是）能够在更复杂的问题上产生更准确的结果。（请注意，对于更简单的基于知识的问题，例如“法国的首都是哪里”，采用这种策略是没有意义的，这同样也是判断推理模型是否适用于给定输入查询的一个很好的经验法则。）

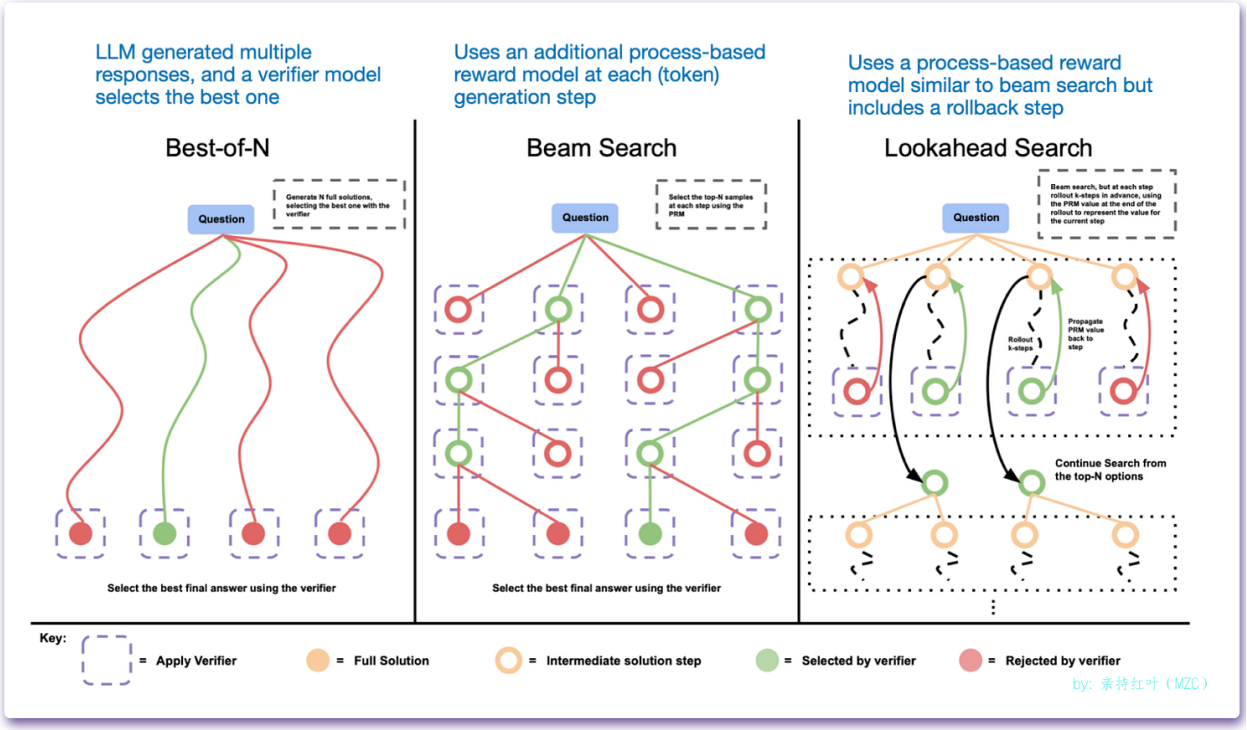


  2022 年大型语言模型零样本推理机论文（https://arxiv.org/abs/2205.11916）中一个经典 CoT 提示的例子。

  前面提到的 CoT 方法可以看作是推理时间扩展，因为它通过生成更多的输出标记使推理更加昂贵。

  另一种提高推理时间可扩展性的方法是使用投票和搜索策略。一个简单的例子是多数投票，我们让 LLM 生成多个答案，然后我们通过多数投票选出正确答案。同样，我们可以使用beam搜索和其他搜索算法来生成更好的答案。

  我强烈推荐我在之前的《2024 年值得关注的 AI 研究论文（第二部分）》（https://magazine.sebastianraschka.com/p/ai-research-papers-2024-part-2）中描述的 Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters]这篇论文，以了解有关这些不同策略的更多详细信息。



DeepSeek R1 技术报告将常见的推理时间扩展方法（例如基于过程奖励模型和蒙特卡洛树搜索的方法）归类为“未成功尝试”。这表明，DeepSeek 并未明确使用这些技术，而只是利用了 R1 模型生成更长响应的自然倾向。与 V3 基础模型相比，R1 模型自然倾向于生成更长的响应，这可以说是一种隐式的推理时间扩展形式。

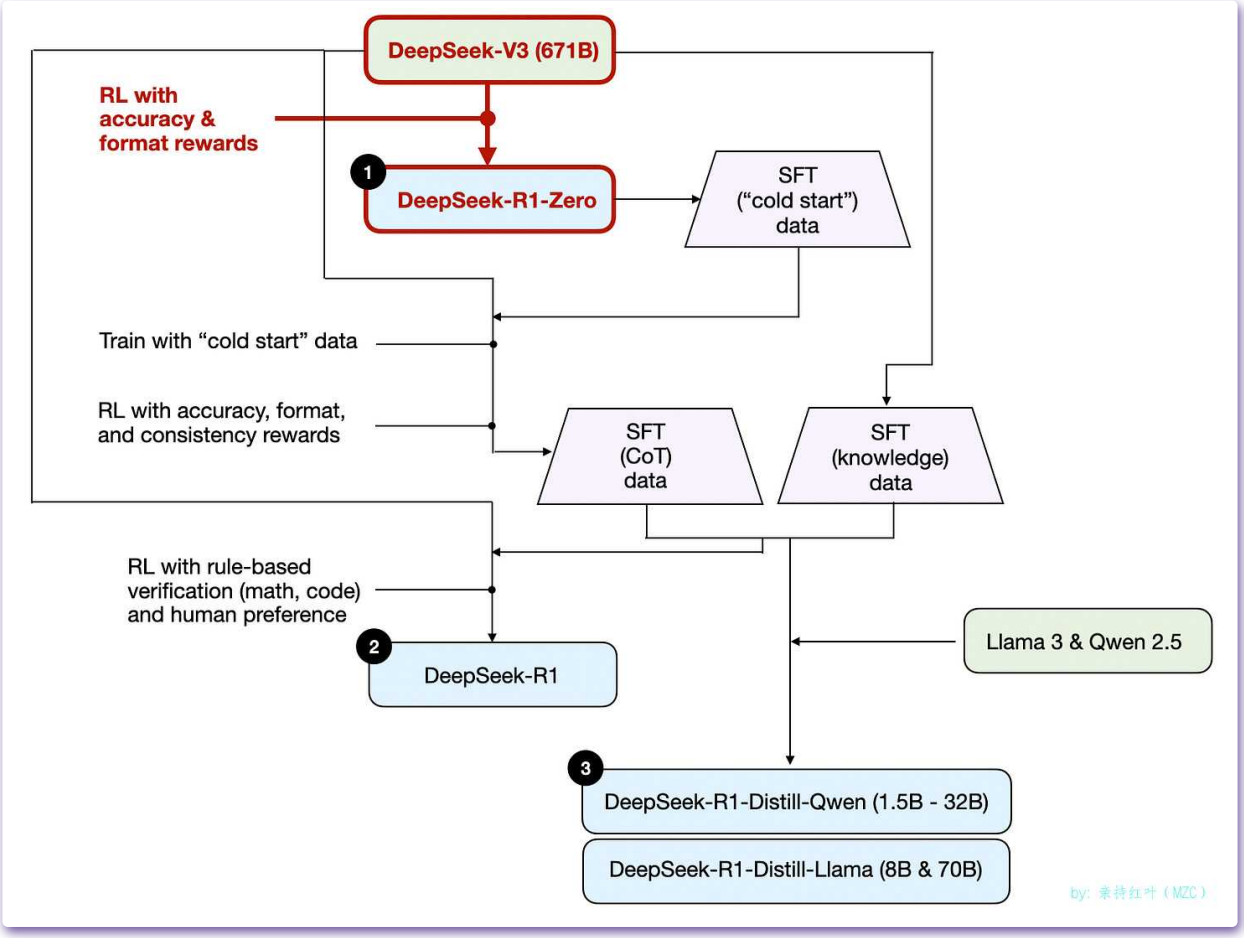
然而，显式推理时间缩放通常是在应用层而不是在 LLM 本身内实现的，因此 DeepSeek 可能仍会在其应用程序中应用此类技术。

我怀疑 OpenAI 的 o1 和 o3 模型使用了推理时间缩放，这可以解释为什么它们与 GPT-4o 等模型相比成本相对较高。除了推理时间缩放之外，o1 和 o3 很可能使用了与 DeepSeek R1 类似的强化学习流程进行训练。更多关于强化学习的内容请参见下面的两节。

1.4.2 Pure reinforcement learning (RL)

- 我个人对 **DeepSeek R1 论文** 的一大亮点是他们发现推理是纯强化学习 (RL) 的一种行为。让我们更详细地探讨一下这意味着什么。

  如前所述，DeepSeek 开发了三种类型的 R1 模型。第一种是 **DeepSeek-R1-Zero** ，它建立在 DeepSeek-V3 基础模型之上，后者是 DeepSeek 于 2024 年 12 月发布的标准预训练 LLM。与典型的强化学习流程（在强化学习之前应用监督微调 (SFT)）不同，DeepSeek-R1-Zero **完全** 采用强化学习进行训练，没有初始的 SFT 阶段，如下图所示。



尽管如此，这种强化学习过程与常用的 RLHF 方法类似，后者通常用于偏好调整LLM。（我在我的文章 **LLM Training: RLHF and Its Alternatives** 中更详细地介绍了 RLHF 。）然而，如上所述， DeepSeek-R1-Zero 的关键区别在于，他们跳过了用于指令调整(instruction)的监督微调 (SFT) 阶段。这就是为什么他们将其称为“纯”强化学习。（不过，LLM 语境下的强化学习与传统强化学习有很大不同，这留待下次再讨论。）

对于奖励，他们没有使用基于人类偏好训练的奖励模型，而是采用了两种类型的奖励：**accuracy reward**(准确性奖励)和 **format reward** (格式奖励)。

1. **准确性奖励**使用 LeetCode complier来验证编码答案，并使用确定性系统来评估数学响应。
2. **格式奖励**依赖于 LLM 评委来确保回答遵循预期格式，例如将推理步骤放在 标签内。

令人惊讶的是，这种方法足以让 LLM 发展出基本的推理能力。研究人员观察到一个“啊哈！”的时刻，尽管模型没有经过明确的训练，但它开始在其反应中生成推理痕迹，如下图所示。

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>
To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$
$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$
Rearrange to isolate the inner square root term:
$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$
$\ldots$
Wait, wait. Wait. That's an aha moment I can flag here.
Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$
We started with the equation:
$\sqrt{a - \sqrt{a + x}} = x$
First, let's square both sides:
$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$
Next, I could square both sides again, treating the equation: $\cdots$
$\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

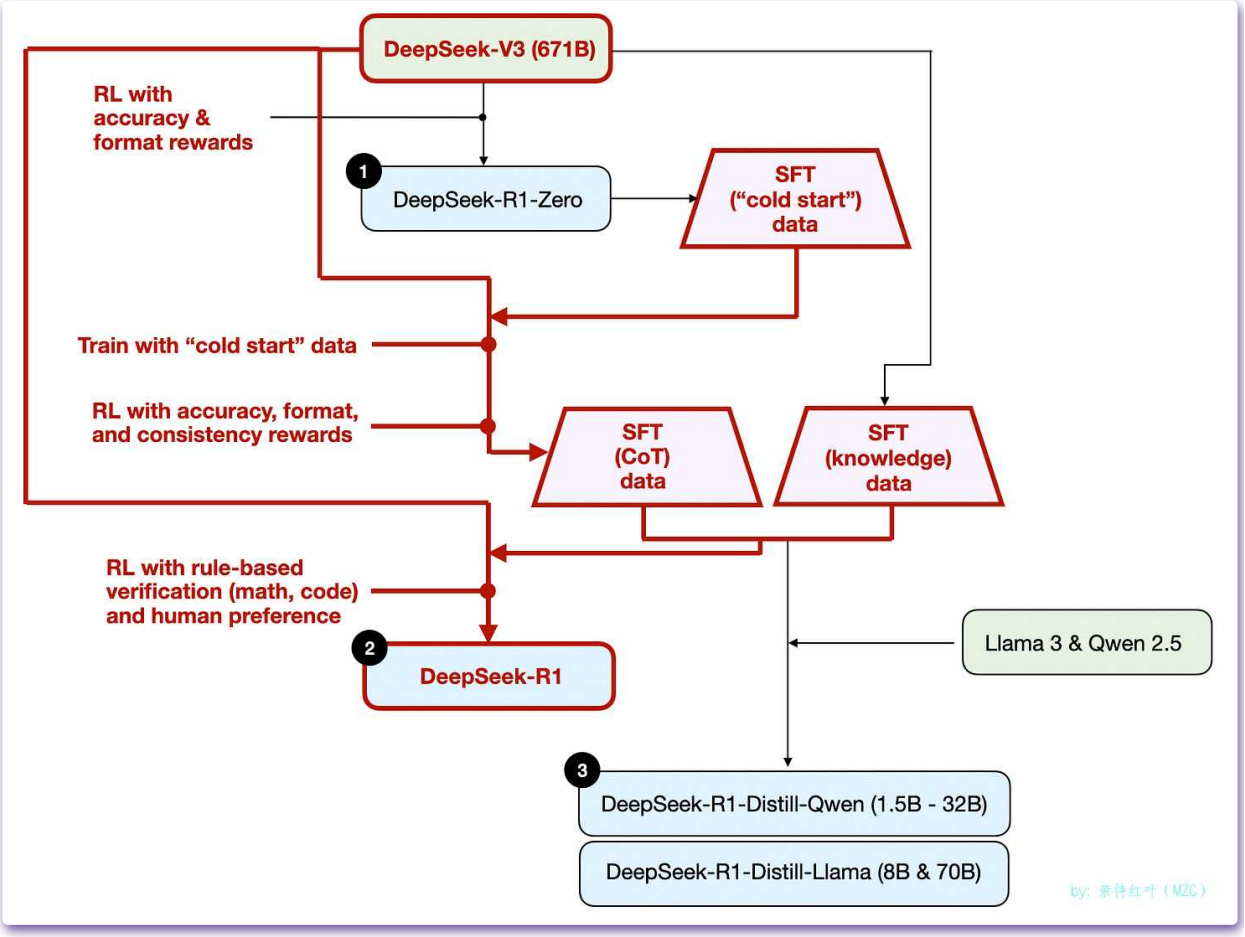
公众: 秉持红叶（MZC）

虽然 R1-Zero 并非性能最佳的推理模型，但它确实通过生成中间的"思考"步骤展示了推理能力，如上图所示。这证实了使用纯强化学习开发推理模型是可行的，而 DeepSeek 团队是第一个展示（或至少是发表）这种方法的团队。

1.4.3 Supervised finetuning and reinforcement learning (SFT + RL)

- 接下来，我们来看一下 DeepSeek 旗舰推理模型 DeepSeek-R1 的开发历程，它是构建推理模型的蓝图。该模型在 DeepSeek-R1-Zero 的基础上进行了改进，加入了额外的监督微调 (SFT) 和强化学习 (RL)，从而提升了其推理性能。

  需要注意的是，在强化学习之前加入一个 SFT 阶段实际上很常见，就像在标准 RLHF 流程中看到的那样。OpenAI 的 o1 很可能就是采用类似的方法开发的。



如上图所示，DeepSeek 团队使用 DeepSeek-R1-Zero 生成了所谓的“冷启动”SFT 数据。“冷启动”指的是这些数据是由 DeepSeek-R1-Zero 生成的，而 DeepSeek-R1-Zero 本身并未接受过任何监督微调 (SFT) 数据的训练（pure RL）。

使用这些冷启动 SFT 数据，DeepSeek 随后通过指令微调训练模型，然后进行另一个强化学习 (RL) 阶段。此 RL 阶段保留了 DeepSeek-R1-Zero RL 流程中使用的准确率和格式奖励。此外，他们还添加了一致性奖励，以防止语言混合（当模型在一次响应中切换多种语言时会发生这种情况）。

强化学习阶段之后，又进行了一轮 SFT 数据收集。在此阶段，我们使用最新的模型检查点生成 60 万个思路链 (CoT) SFT 示例，同时使用 DeepSeek-V3 基础模型创建了另外 20 万个基于通用知识的 SFT 示例。

这 60 万 + 20 万个 SFT 样本随后被用于指令微调 DeepSeek-V3 基础模型，之后进行最后一轮强化学习。在此阶段，他们再次使用基于规则的方法对数学和编程问题进行准确率奖励，而对其他问题类型则使用人类偏好标签。总而言之，这与常规的 RLHF 非常相似，只是 SFT 数据包含（更多）CoT 示例。而且，除了基于人类偏好的奖励之外，强化学习还具有可验证的奖励。

- 最终模型 DeepSeek-R1 由于增加了 SFT 和 RL 阶段，性能较 DeepSeek-R1-Zero 有显著提升，如下表所示。

	Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
		pass@1	cons@64	pass@1	pass@1	pass@1	rating
	OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
	OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
RL only	DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444
SFT + RL	**DeepSeek-R1**	**79.8**		**97.3**	**71.5**	**65.9**	**2029**

Math benchmarks | Bio, physics & chemistry | Code benchmarks

Higher is better

by: 亲持红叶（MZC）

1.4.4 Pure supervised finetuning (SFT) and distillation

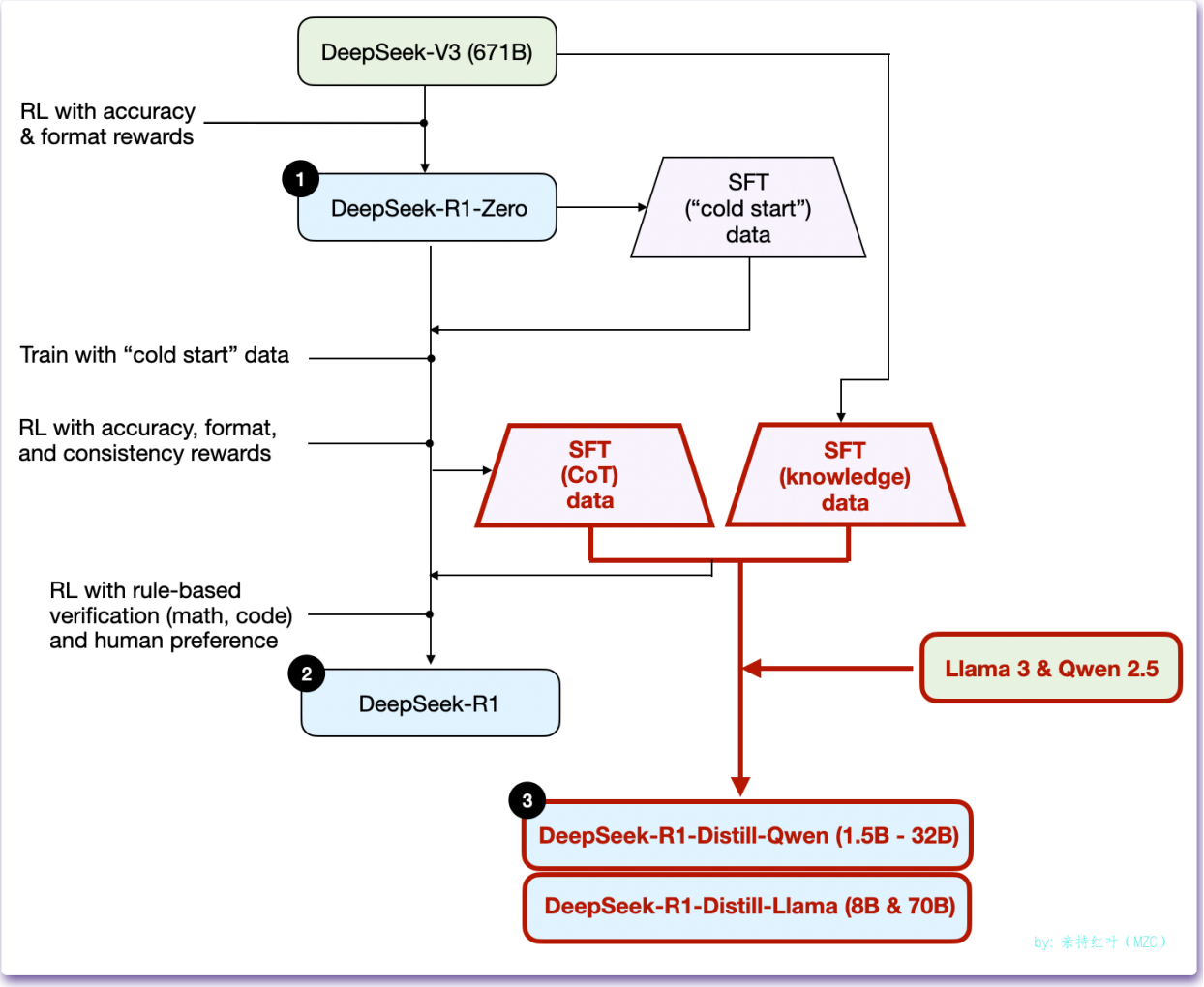
- 到目前为止，我们已经介绍了构建和改进推理模型的三种主要方法：
    1. **Inference-time scaling**（推理时间扩展），一种无需训练或修改底层模型即可提高推理能力的技术。
    2. **Pure reinforcement learning**（纯强化学习），如 DeepSeek-R1-Zero，它表明推理可以作为一种学习行为出现，而无需监督微调。
    3. **Supervised fine-tuning**（监督微调sft）加上 RL，从而产生了 DeepSeek 的旗舰推理模型 DeepSeek-R1。

  那么，还剩下什么呢？模型“蒸馏”。

- 令人惊讶的是，DeepSeek 还发布了一些规模较小的模型，这些模型是通过他们称之为 “知识 *蒸馏”的* 过程训练的 。然而，LLM的背景下，知识蒸馏并不一定遵循深度学习中使用的经典知识蒸馏方法。传统上，在知识蒸馏中（正如我在《[Machine Learning Q and AI 》一书第六章中简要描述的那样），一个规模较小的学生模型会同时在规模较大的教师模型和目标数据集的logits上进行训练。

  而此处的蒸馏指的是在由较大 LLM 生成的 SFT 数据集上，对较小的 LLM（例如 Llama 8B 和 70B 以及 Qwen 2.5 模型（0.5B 到 32B））进行指令微调。具体来说，这些较大的 LLM 是 DeepSeek-V3 和 DeepSeek-R1 的一个中间检查点。实际上，此蒸馏过程使用的 SFT 数据与上一节中描述的用于训练 DeepSeek-R1 的数据集相同。

  为了阐明这一过程，我在下图中突出显示了蒸馏部分。



他们为什么要开发这些提炼模型？我认为有两个主要原因：

1. 更小的模型效率更高。这意味着它们运行成本更低，而且也能在低端硬件上运行，这对于像我这样的许多研究人员和爱好者来说尤其有趣。
2. 纯监督微调 (SFT) 案例研究。这些经过提炼的模型可以作为有趣的基准，展示纯监督微调 (SFT) 在没有强化学习的情况下，模型可以走多远。

下表比较了这些提炼模型与其他流行模型以及 DeepSeek-R1-Zero 和 DeepSeek-R1 的性能。

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	**1820**
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	**72.6**	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	**86.7**	**94.5**	**65.2**	**57.5**	1633
DeepSeek-R1-Zero	**71.0**		**95.9**	**73.3**	**50.0**	**1444**
DeepSeek-R1	**79.8**		**97.3**	**71.5**	**65.9**	**2029**

by: 亲持红叶（MZC）

正如我们所看到的，蒸馏模型明显弱于 DeepSeek-R1，但相对于 DeepSeek-R1-Zero 来说，它们却出奇地强大，尽管它们要小几个数量级。与 o1 mini 相比，这些型号的性能也很有趣（我怀疑 o1-mini 本身可能是 o1 的类似蒸馏版本)。

- 在用结论结束本节之前，还有一个有趣的比较值得一提。DeepSeek 团队测试了 DeepSeek-R1-Zero 中出现的涌现推理行为是否也能在较小的模型中出现。为了验证这一点，Qwen团队将 DeepSeek-R1-Zero 中相同的纯强化学习方法直接应用于 Qwen-32B。

  本次实验的结果总结如下表所示，其中 QwQ-32B-Preview 是 Qwen 团队基于 Qwen 2.5 32B 开发的参考推理模型（我认为训练细节从未公开过）。本次比较提供了一些额外的见解，让我们了解纯强化学习能否在比 DeepSeek-R1-Zero 小得多的模型中诱导推理能力。

	Model	AIME 2024		MATH-500	GPQA Diamond	LiveCodeBench
		pass@1	cons@64	pass@1	pass@1	pass@1
	QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9
pure RL →	DeepSeek-R1-Zero-Qwen-32B	47.0	60.0	91.6	55.0	40.2
	DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2
pure RL →	DeepSeek-R1-Zero	71.0		95.9	73.3	50.0
	DeepSeek-R1	79.8		97.3	71.5	65.9

by: 秉持红叶（MZC）

  有趣的是，结果表明，对于较小的模型，蒸馏比纯强化学习更有效。这与以下观点相符：单靠强化学习可能不足以在这种规模的模型中诱导出强大的推理能力，而使用高质量推理数据进行随机场推理 (SFT) 则可能在处理小型模型时成为一种更有效的策略。

  为了完整性，查看表格中的其他比较会很有用：

  1. Qwen-32B 采用 SFT + RL 进行训练，类似于 DeepSeek-R1 的开发方式。这将有助于确定当 RL 与 SFT 结合时，相比纯 RL 和纯 SFT，可以实现多大的改进
  2. DeepSeek-V3 使用纯 SFT 进行训练，与蒸馏模型的创建方式类似。这样可以直接比较强化学习 + SFT 相对于纯 SFT 的效果。

1.4.5 Conclusion

- 在本节中，我们探讨了构建和改进推理模型的四种不同策略：
  1. 推理时间扩展无需额外训练，但会增加推理成本，随着用户数量或查询量的增长，大规模部署的成本也会更高。尽管如此，对于提升原本就很强大的模型的性能来说，这仍然是一个显而易见的选择。我强烈怀疑 o1 利用了推理时间扩展，这有助于解释为什么它比 DeepSeek-R1 的单位 token 成本更高。
  2. 纯强化学习在研究方面很有意思，因为它能洞察推理作为一种新兴行为。然而，在实际的模型开发中，RL+SFT 才是首选方法，因为它能构建更强大的推理模型。我强烈怀疑 o1 也是用 RL + SFT 训练的。更准确地说，我认为 o1 的基础模型比 DeepSeek-R1 更弱、更小，但通过 RL + SFT 和推理时间扩展弥补了这一缺陷。
  3. 如上所述，RL + SFT 是构建高性能推理模型的关键方法。DeepSeek-R1 是一个很好的蓝图，展示了如何实现这一点。
  4. 蒸馏是一种颇具吸引力的方法，尤其是在创建更小、更高效的模型时。然而，蒸馏的局限性在于它无法推动创新，也无法产生下一代推理模型。例如，蒸馏

总是依赖于现有的、更强大的模型来生成监督微调 (SFT) 数据。

我预计接下来会看到一个有趣的方面，那就是将 RL + SFT（方法 3）与推理时间扩展（方法 1）结合起来。OpenAI o1 很可能就是这么做的，只不过它的基础模型可能比 DeepSeek-R1 更弱，这也解释了为什么 DeepSeek-R1 表现如此出色，同时推理时间成本却相对低廉。

1.5 Thoughts about DeepSeek R1

- 最近几周，很多人问我对 DeepSeek-R1 模型的看法。简而言之，我认为它们是一项了不起的成就。作为一名研究工程师，我尤其欣赏这份详细的技术报告，它深入分析了他们的方法论，值得我借鉴。

  最引人入胜的发现之一是，推理是如何从纯强化学习中发展成为一种行为的。令人印象深刻的是，DeepSeek 已根据宽松的 MIT 开源许可证开源了他们的模型，该许可证的限制甚至比 Meta 的 Llama 模型还要少。

1.5.1 How does it compare to o1?

- DeepSeek-R1 比 o1 更好吗？我认为两者大致相同。然而，最突出的是 DeepSeek-R1 在推理时间上更高效。这表明 DeepSeek 可能在训练过程中投入了更多，而 OpenAI 可能在 o1 上更依赖于推理时间的扩展。

  话虽如此，直接比较 o1 和 DeepSeek-R1 仍然很困难，因为 OpenAI 尚未披露太多关于 o1 的信息。例如，我们不知道：

  1. o1 也是专家混合体 (MoE) 吗？
  2. o1 有多大？
  3. o1 是否可能只是 GPT-4o 的稍微改进版本，具有最少的 RL + SFT 和仅广泛的推理时间缩放？

  如果不知道这些细节，直接的比较就只是苹果和橘子的比较。

1.5.2 The cost of training DeepSeek-R1

- 另一个讨论点是 DeepSeek-R1 的开发成本。有人提到训练成本约为 600 万美元，但他们很可能把 DeepSeek-V3（去年 12 月发布的基础模型）和 DeepSeek-R1 混为一谈了。

  600 万美元的估算是基于每 GPU 小时 2 美元的假设以及 DeepSeek-V3 最终训练运行所需的 GPU 小时数，该估算最初于 2024 年 12 月进行讨论。

  然而，DeepSeek 团队从未透露过 R1 的具体 GPU 小时数或开发成本，因此任何成本估算都只是纯粹的猜测。

  最终，无论如何DeepSeek-R1 是开放权重推理模型的一个重要里程碑，其推理时间的效率使其成为 OpenAI 的 o1 的一个有趣的替代品。

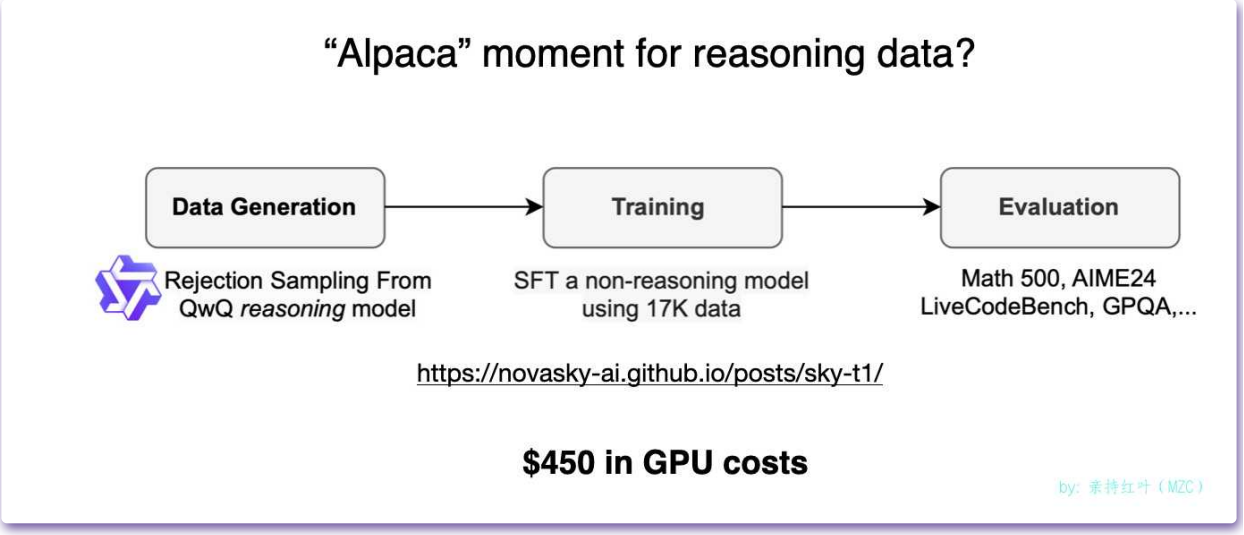
1.6 Developing reasoning models on a limited budget

- 开发一个 DeepSeek-R1 级别的推理模型可能需要数十万甚至数百万美元，即使从像 DeepSeek-V3 这样的开放权重基础模型开始也如此。这对于预算有限的研究人员或工程师来说，可能会令人望而却步。
- **好消息：蒸馏可以发挥很大作用**

  幸运的是，模型蒸馏提供了一种更具成本效益的替代方案。DeepSeek 团队通过他们的 R1 蒸馏模型证明了这一点，尽管模型规模远小于 DeepSeek-R1，但却实现了令人惊讶的强大推理性能。然而，即使是这种方法也并非完全便宜。他们的蒸馏过程使用了 80 万个 SFT 样本，这需要大量的计算。

  有趣的是，就在 DeepSeek-R1 发布的几天前，我偶然看到了 **一篇关于 Sky-T1 的文章** 。这是一个引人入胜的项目，一个小团队仅使用 17000 个 SFT 样本训练了一个开放权重的 32B 模型。总成本是多少？只需 450 美元，比大多数 AI 会议的注册费还低。

  这个例子强调了，虽然大规模训练仍然昂贵，但规模较小、有针对性的微调工作仍然可以以极低的成本产生令人印象深刻的结果。



  根据他们的基准测试，Sky-T1 的表现与 o1 大致相当，考虑到其低廉的训练成本，这一表现令人印象深刻。

- **预算有限的纯强化学习：TinyZero**

  虽然 Sky-T1 专注于模型蒸馏，但我也发现了一些"纯强化学习"领域的有趣工作。其中一个值得注意的例子是 **TinyZero** ，它是一个 3B 参数模型，复制了 DeepSeek-R1-Zero 的方法（附注：它的训练成本不到 30 美元）。

  令人惊讶的是，即使只有 3B 个参数，TinyZero 也表现出一些新兴的自我验证能力，这支持了推理可以通过纯 RL 出现的想法，即使在小模型中也是如此。

  TinyZero 存储库 提到研究报告仍在进行中，我一定会密切关注更多细节。

User: Using the numbers [19, 36, 55, 7], create an equation that equals 65.
Assistant: Let me solve this step by step.
<think> We need to find an equation using the numbers 19, 36, 55, and 7 exactly once, with basic arithmetic operations, that equals 65. One possible combination is 55 + 36 - 19 + 7. Let's check: 55 + 36 = 91, 91 - 19 = 72, and 72 + 7 = 79. However, this doesn't equal 65. Another combination is 55 + 36 + 7 - 19. Let's check: 55 + 36 = 91, 91 + 7 = 98, and 98 - 19 = 79. This also doesn't equal 65. After trying different combinations, I found that 55 + 36 + 7 - 19 = 79, which is close but not equal to 65. Finally, I found that 55 + 36 - 19 + 7 = 79, which is still not equal to 65. However, if we try 55 + 36 - 7 + 19, we get 55 + 36 = 91, 91 - 7 = 84, and 84 + 19 = 103, which is not equal to 65. After further trial and error, I found that 55 + 36 + 7 - 19 = 79, which is still not equal to 65. However, if we try 55 + 36 - 7 - 19, we get 55 + 36 = 91, 91 - 7 = 84, and 84 - 19 = 65. This equals 65. </think>
<answer> 55 + 36 - 7 - 19 </answer>

by: 秉持红叶（MZC）

上述是TinyZero 代码库（https://github.com/Jiayi-Pan/TinyZero）中的一张图，显示该模型能够进行自我验证。（如果能比较一下基础模型的响应，应该会很有趣。）

上述两个项目表明，即使预算有限，在推理模型方面也能开展一些有趣的工作。虽然这两种方法都复制了 DeepSeek-R1 的方法，但其中一个专注于纯RL (TinyZero)，另一个专注于纯SFT (Sky-T1)，探索如何进一步扩展这些想法将会非常有趣。

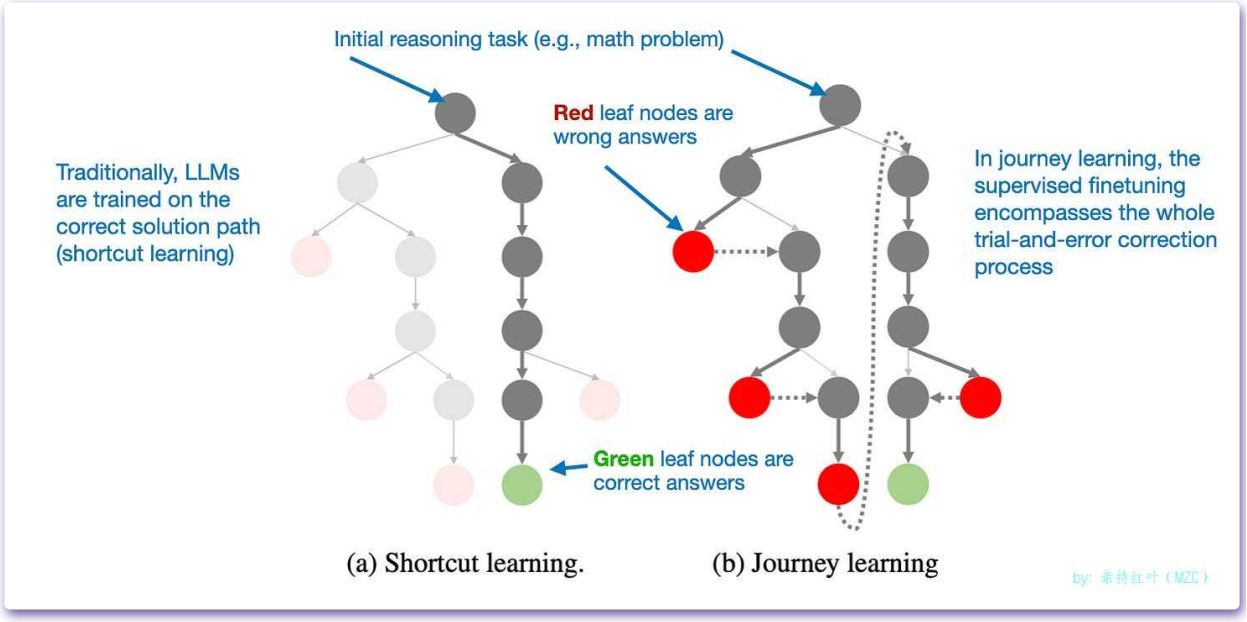
- **超越传统 SFT：Journey Learning（旅程学习）**

我去年偶然发现一个特别有趣的方法，发表在《O1 Replication Journey: A Strategic Progress Report – Part 1》这篇论文中。尽管标题如此，但这篇论文实际上并没有复制 o1。相反，它介绍了一种改进蒸馏（纯 SFT）工艺的另一种方法。

论文的核心思想是用“journey learning”来替代“shortcut learning”。

1. Shortcut learning是指指令微调中的传统方法，其中仅使用正确的解决方案路径来训练模型。
2. Journey learning包括错误的解决路径，让模型从错误中学习。

这种方法与 TinyZero 在纯强化学习训练中观察到的自我验证能力有些相似，但它专注于完全通过 SFT 来改进模型。通过让模型接触错误的推理路径及其修正，Journey learning或许还能增强自我修正能力，从而有可能通过这种方式提高推理模型的可靠性。



这可能是未来工作的一个令人兴奋的方向，特别是对于低预算推理模型开发，其中基于 RL 的方法在计算上可能不切实际。

无论如何，推理模型方面目前正在进行许多有趣的工作，我相信我们将在接下来的几个月里看到更多令人兴奋的工作！